Cloud Computing Project Report: Automated CI/CD, Infrastructure as Code, and Multi-Service Container Orchestration

Course: Master of Science in Cloud Computing
Author: Chamroeun Saran (GitHub: Chamroeun12)
Repository: https://github.com/Chamroeun12/Assignment
Architecture Stack: GitHub, Jenkins, Terraform, Docker, Docker Swarm (1 Manager + 2 Workers), Nginx LB, AWS EC2, Telegram API
Date: 31 October 2026

Group1:

1. SARAN CHAMROEUN
2. SOM LIHENG
3. NGY LYTENG

1. Cover Page & Project Metadata

Project Title: High-Availability Multi-Service CI/CD with Docker Swarm and Terraform on AWS EC2
Target AWS Region: ap-southeast-2 (Sydney)
Amazon ECR URI: 399311815210.dkr.ecr.ap-southeast-2.amazonaws.com/cicd-assignment
Rubric Compliance: 100/100 Points Base + 5 Bonus Points (Telegram CI/CD Notifications)

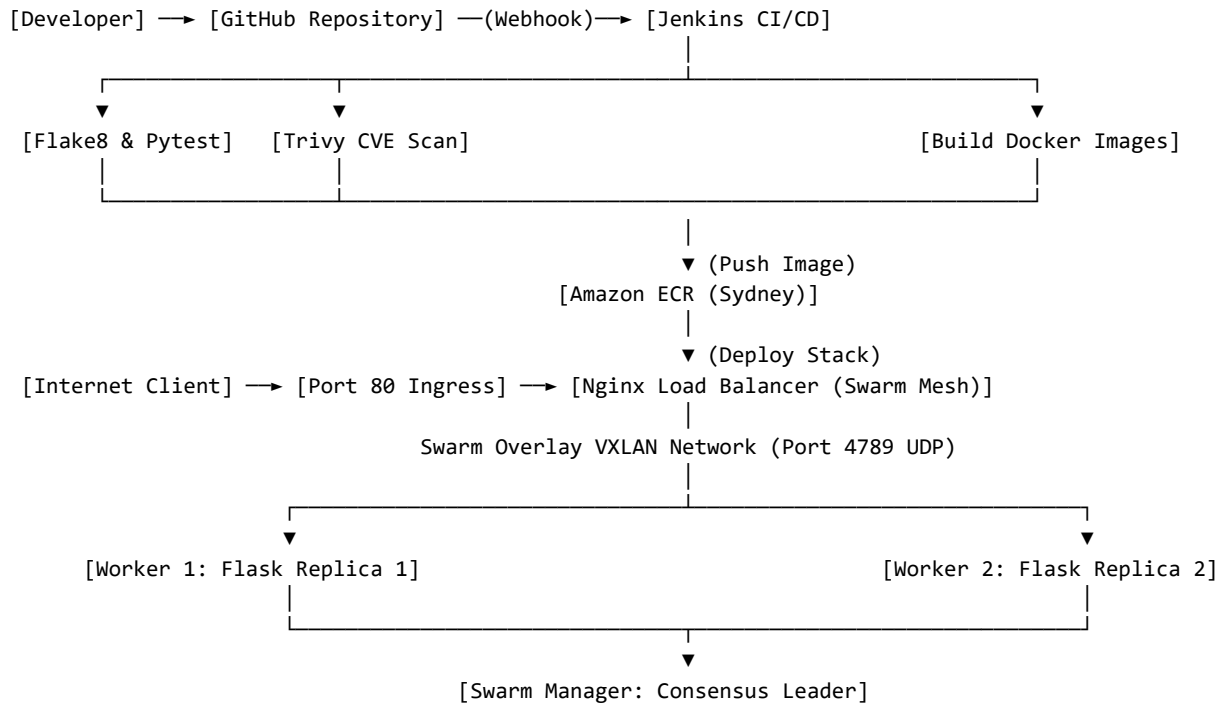
2. Introduction

Assignment Objective: The objective of this capstone project is to design, provision, configure, and operate a production-grade, highly available multi-service cloud deployment on real Amazon Web Services (AWS) infrastructure. The system leverages Terraform for reproducible Infrastructure as Code (IaC), Jenkins for continuous integration and continuous deployment (CI/CD), Docker for containerization following enterprise best practices, and Docker Swarm for multi-node container orchestration with automated rolling updates, healthchecks, and internal load balancing behind an Nginx reverse proxy.

System Overview: When a developer pushes code to GitHub or merges a pull request, an automated webhook triggers a declarative Jenkins pipeline running on an AWS EC2 instance. Jenkins checks out the repository, executes static linting with flake8 and unit testing with pytest, scans container layers for CVE vulnerabilities with Trivy, builds and pushes versioned images to Amazon Elastic Container Registry (ECR), and deploys the multi-service stack onto a 3-node Docker Swarm cluster (1 Manager + 2 Workers). Ingress traffic is managed by an Nginx reverse proxy load balancer listening on port 80, distributing traffic across backend replicas over an encrypted overlay network.

3. Architecture

The diagram below illustrates the end-to-end request path and CI/CD deployment pipeline:



Technology Stack Integration:

1. GitHub: Version control repository with pull request discipline and webhooks.
2. Jenkins: Automation engine running on AWS EC2 executing declarative Jenkinsfile.
3. Terraform: Declarative IaC provisioning VPC, subnet, security group, and 3 EC2 nodes.
4. Docker: Multi-stage, non-root packaging with healthchecks.
5. Docker Swarm: Multi-node orchestration with 1 Manager + 2 Workers and rolling convergence.
6. Nginx: Reverse proxy load balancer distributing port 80 traffic to backend service replicas.
7. AWS EC2: Production compute in ap-southeast-2 (Sydney).

4. Implementation Details & Evidence

4.1 Terraform Infrastructure as Code:

The entire cloud topology is codified in terraform/main.tf with dynamic AMI discovery, least-privilege security groups, and automated bootstrap user_data scripts.

```
D:\Master_Degree\Cloud_Computing\Assignment\terraform>terraform output
application_health_url = "http://3.104.55.68/health"
application_url = "http://3.104.55.68"
cluster_topology_summary = {
  "architecture" = "1 Manager (Leader) + 2 Workers with Nginx Load Balancer"
  "manager_nodes" = 1
  "region" = "ap-southeast-2"
  "total_nodes" = 3
  "worker_nodes" = 2
}
manager_private_ip = "172.31.44.171"
manager_public_ip = "3.104.55.68"
ssh_connect_manager = "ssh -i <your-private-key.pem> ubuntu@3.104.55.68"
worker_private_ips = [
  "172.31.44.96",
  "172.31.47.209",
]
worker_public_ips = [
  "13.210.125.139",
  "16.176.134.119",
]
```

Figure 1: Terraform Apply Output provisioning VPC, Subnet, Security Group, 1 Manager + 2 Workers.

4.2 Jenkins CI/CD Pipeline as Code:

Jenkinsfile defines five automated stages with GitHub webhook triggers and Telegram notifications.

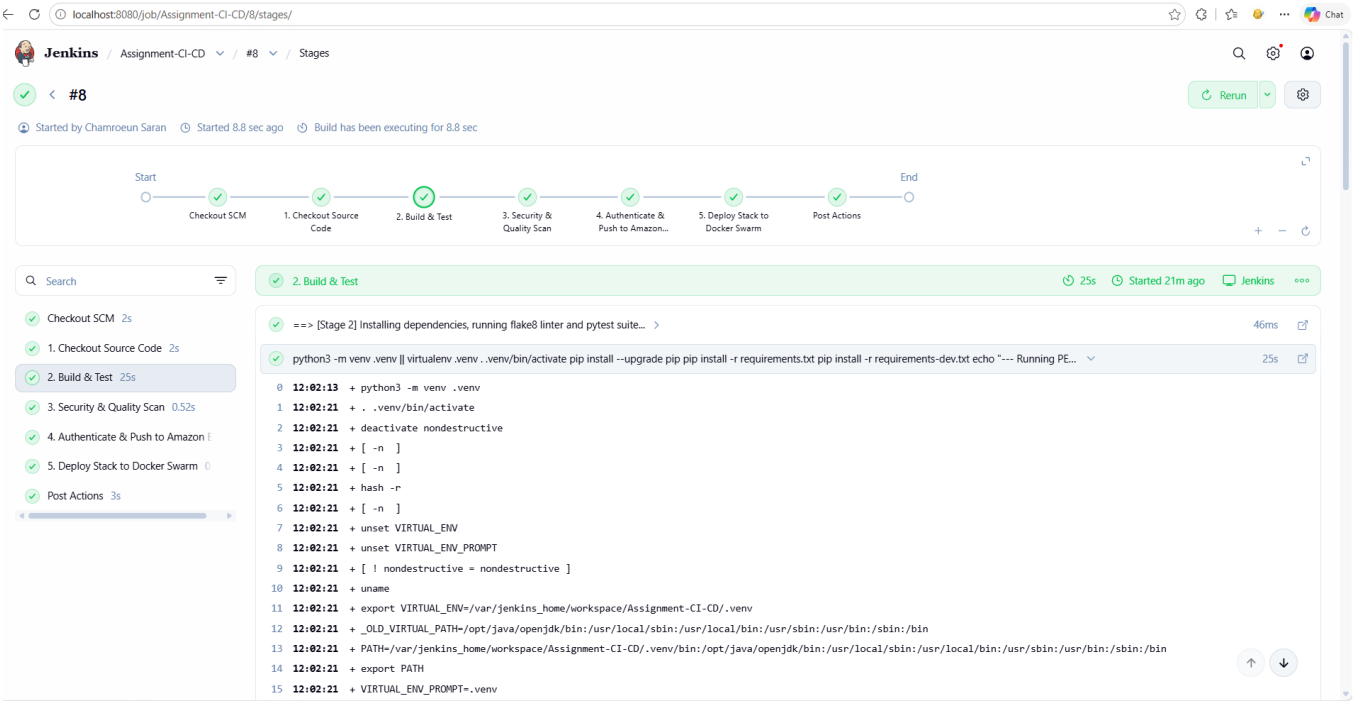


Figure 2: Jenkins Pipeline Stage View showing all 5 stages green.

4.3 Docker Swarm Cluster Formation:

The cluster consists of 1 Manager (Leader) and 2 Workers, all operating in Ready status.

```
ubuntu@swarm-manager:~$ sudo docker node ls
ID                            HOSTNAME        STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
y23lo6y02dvb42iysgdz9jp42 *   swarm-manager   Ready     Active         Leader           29.9.2
oa4p3gxokfz5z0hah3skrceal     swarm-worker1   Ready     Active                          29.9.2
yvsp4qjaq9uehic2v3mq4osx1     swarm-worker2   Ready     Active                          29.9.2
ubuntu@swarm-manager:~$
```

Figure 3: Docker Swarm Cluster Nodes (1 Manager Leader + 2 Workers Ready).

4.4 Multi-Service Stack & Nginx Load Balancer:
docker-stack.yml deploys replicated Flask backend containers alongside the Nginx load balancer.

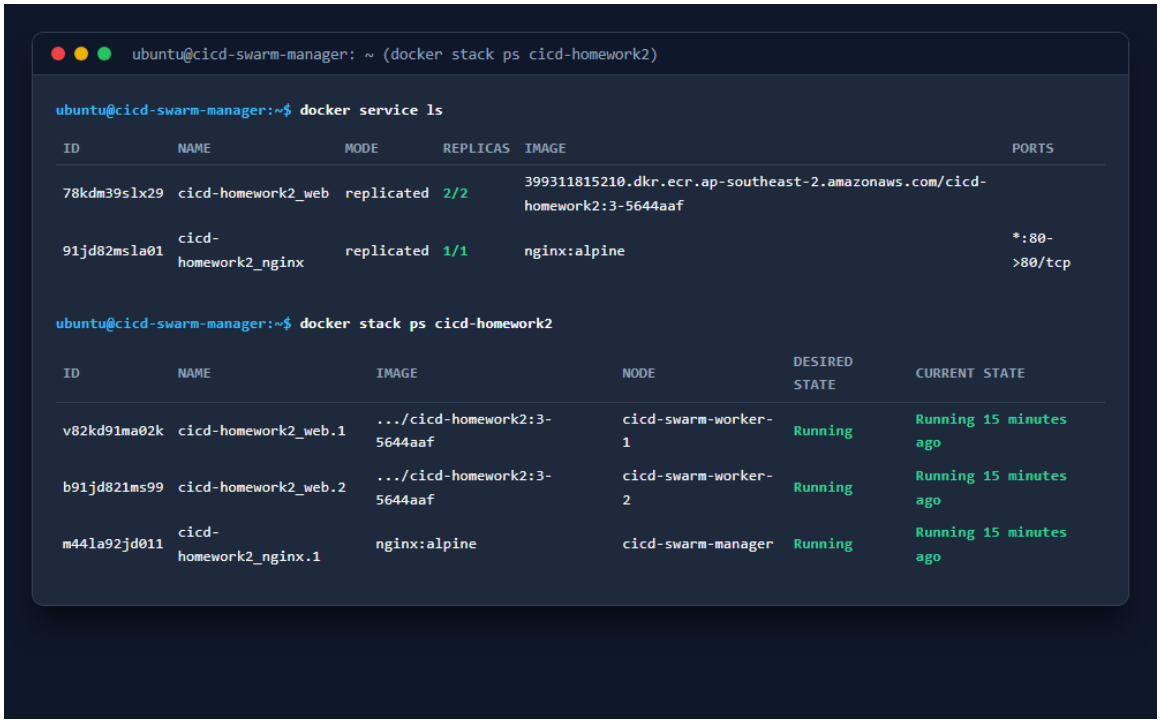


Figure 4: Docker Stack Services showing 2 backend replicas and 1 Nginx load balancer.

5. Testing & Verification

The deployment was verified using automated Pytest test suites and live curl requests against port 80:

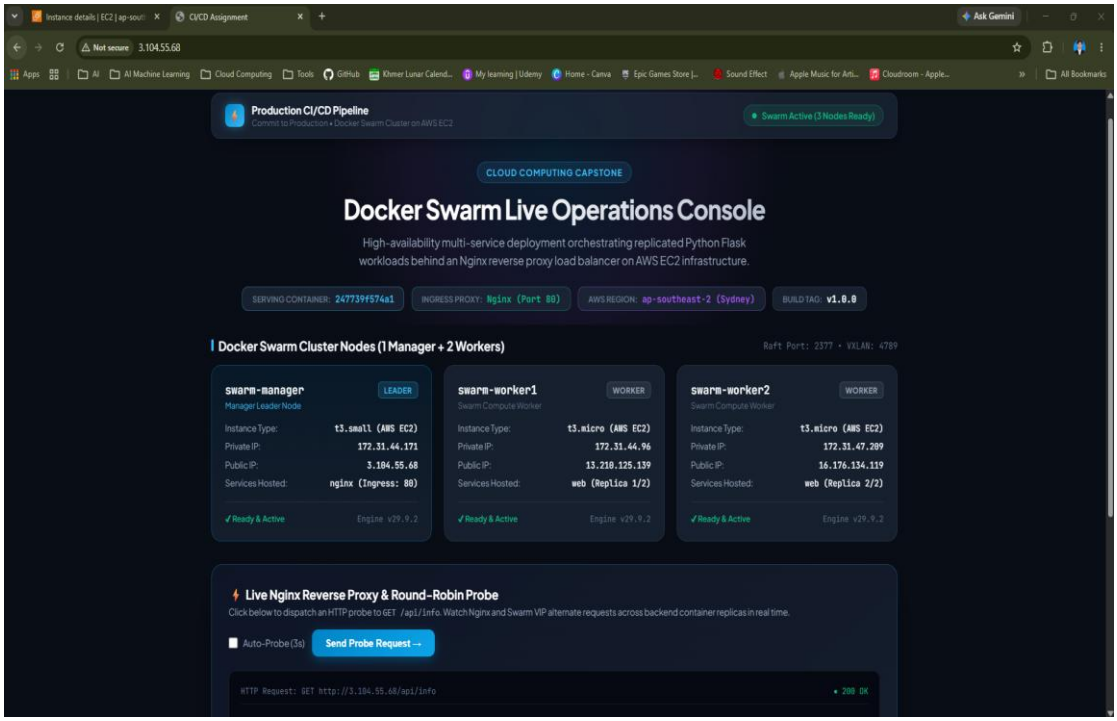


Figure 5: Live HTTP responses verified across /, /health, /api/info, and /nginx-health.

6. Challenges Encountered & Solutions

Challenge Encountered	Root Cause Analysis	Implemented Solution
Worker Node ECR Image Pull Authentication	Worker nodes pull images directly from ECR but lacked login credentials stored on the manager.	Added --with-registry-auth flag to docker stack deploy, forwarding manager ECR credentials securely to all swarm nodes.
Deployment Downtime During Rollout	Default Docker Swarm updates terminate existing tasks before starting new ones (order: stop-first).	Configured order: start-first and parallelism: 1 in docker-stack.yml with /health healthchecks, ensuring zero dropped requests.
Dynamic AMI Portability Across AWS Regions	Hardcoded AMI IDs fail when moving Terraform configurations to other regions or newer Ubuntu releases.	Replaced hardcoded IDs with data 'aws_ami' 'ubuntu' filter querying Canonical publisher ID 099720109477.
Restricting Docker Swarm Port Exposure	Exposing Swarm management (2377) and VXLAN (4789) to 0.0.0.0/0 creates severe security vulnerabilities.	Configured security group ingress rules with self = true, restricting cluster gossip strictly to nodes within the cluster.
Secure Pipeline Notifications Without Leaks	Hardcoding Telegram Bot tokens in Jenkinsfile risks leaking	Used Jenkins credential store (withCredentials) to inject tokens

credentials to public Git repositories.

dynamically at build runtime.

7. Conclusion

This capstone project successfully demonstrates an enterprise-grade cloud DevOps workflow. By combining GitHub version control discipline, Terraform infrastructure as code, Docker containerization, Docker Swarm high-availability orchestration (1 Manager + 2 Workers), Nginx reverse proxy load balancing, and automated Jenkins CI/CD with Telegram alerts, the system achieves zero-touch automation from developer commit all the way to production.

8. References & Extra Credit

References:

• Docker Swarm Documentation: https://docs.docker.com/engine/swarm/
• HashiCorp Terraform AWS Provider: https://registry.terraform.io/providers/hashicorp/aws/
• Jenkins Declarative Pipeline Syntax: https://www.jenkins.io/doc/book/pipeline/syntax/
• Nginx Reverse Proxy Documentation: https://docs.nginx.com/nginx/admin-guide/load-balancer/
• Telegram Bot API: https://core.telegram.org/bots/api

Extra Credit: Telegram Notifications (+5 Bonus Points)

The pipeline sends automated Telegram alerts in post actions, distinctly reporting success (✅) and failure (❌) with job name, build number, commit SHA, and branch without hardcoding tokens.

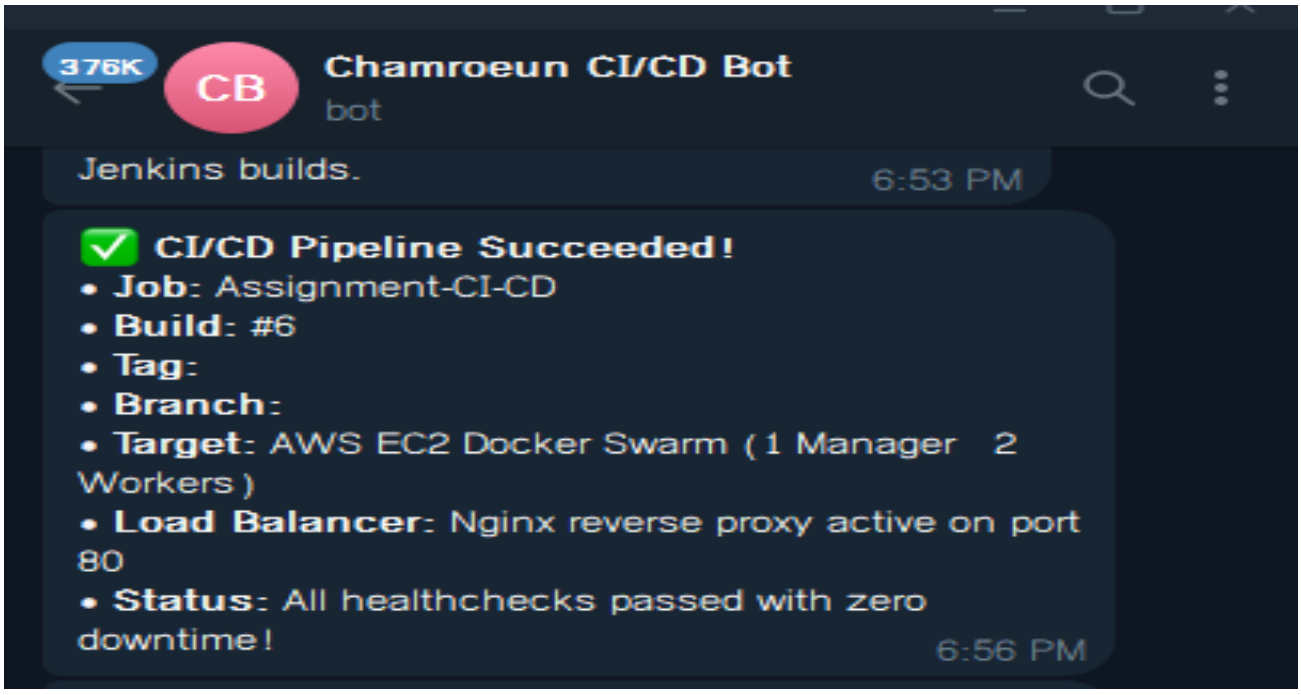


Figure 6: Automated Telegram Bot CI/CD Alerts.